Project Report

x86 32-bit Assembly – MASM32

Recursive Directory Explorer (CLI and GUI)

O. Denis

[University]

February 2026

1 Introduction

This project is a recursive directory explorer written in x86 32-bit assembly using MASM32. Two versions were implemented: a Command-Line Interface (CLI) and a Windows Graphical User Interface (GUI). Both implementations traverse a directory tree iteratively using a static stack, avoiding native language recursion (no recursive function calls).

2 CLI Version

2.1 Architecture

2.1.1 `.data` segment (initialised data)

The `.data` segment holds the strings needed for display and file-search operations:

- `adr1B`: prompt shown to the user (`"Path:  "`)
- `msgDirectory`: header printed before each visited directory
- `searchPattern`: the `\*.*` wildcard pattern passed to `FindFirstFileA`

2.1.2 `.data?` segment (uninitialised data)

- `path`: user-entered path (260 bytes: 256 + drive letter + colon + slash + null)
- `searchPath`: path concatenated with `\*.*` for `FindFirstFileA`
- `findData`: `WIN32_FIND_DATAA` structure (320 bytes)
- `stackPaths`: static path stack (60×260 bytes $= 15\,600$ bytes)
- `stackPtr`: stack top pointer (index of the next free slot)
- `currentPath`, `tempBuffer`: working buffers for path concatenation

Using a static stack prevents call-stack overflows. The chosen maximum depth of 60 levels covers the vast majority of real Windows directory trees.

2.2 How it works

2.2.1 Acquiring console handles

```
push -11            ; STD_OUTPUT_HANDLE constant
call GetStdHandle   ; kernel32.dll -- returns handle in eax
mov stdOut, eax     ; save stdout handle

push -10            ; STD_INPUT_HANDLE constant
call GetStdHandle
mov stdIn, eax      ; save stdin handle
```

`GetStdHandle` is called with constant `-11` (stdout) and `-10` (stdin). The returned value in `eax` is saved to memory. All display (`WriteFile`) and read (`ReadFile`) operations require a valid handle.

2.2.2 Reading and cleaning the path

After `ReadFile` reads the path, the trailing 2 bytes (CR LF) are stripped:

```
mov ecx, bytesRead            ; byte count returned by ReadFile
sub ecx, 2                    ; remove CR LF
mov byte ptr [path + ecx], 0  ; null-terminate the string
mov pathLength, ecx           ; save effective length
```

2.3 Main loop

The `boucle_stack` loop pops one path, lists the contents of the corresponding directory, pushes any subdirectories found, and repeats until the stack is empty.

```
boucle_stack:
    mov eax, stackPtr
    cmp eax, 0              ; stack empty?
    jz fin_recursion        ; yes -- all directories processed

    sub stackPtr, 260       ; pop: move pointer back one slot
    mov eax, stackPtr
    lea esi, [stackPaths + eax]
    mov edi, offset currentPath
    mov ecx, 260            ; copy at most 260 bytes
    ; ... (byte-by-byte copy loop)
    jmp boucle_stack
```

2.4 Subdirectory detection

The `FILE_ATTRIBUTE_DIRECTORY` flag (bit 4 = `0x10`) is tested in the first DWORD of `WIN32_FIND_DATAA`:

```
mov eax, dword ptr findData ; dwFileAttributes (offset 0)
and eax, 10h                ; mask directory bit
jz pas_dossier              ; flag absent => regular file
```

Note: reading via the symbolic label `dword ptr findData` (rather than a compile-time resolved direct address) was necessary to prevent corruption of the generated `.obj` file during early testing.

2.5 Filtering `.` and `..`

The `.` (current directory) and `..` (parent directory) entries are always present in every Windows directory. Without filtering, the loop would be infinite:

```
lea esi, [findData+44]    ; cFileName at offset 44 in
    WIN32_FIND_DATAA
mov al, byte ptr [esi]
cmp al, '.'
jne pas_point             ; first char != '.' -- process normally

```

```
mov al, byte ptr [esi+1]
cmp al, 0                   ; is it just "." ?
je fichier_suivant          ; skip

cmp al, '.'
jne pas_point

mov al, byte ptr [esi+2]
cmp al, 0                   ; is it just ".." ?
je fichier_suivant          ; skip
```

3 GUI Version

3.1 Architecture

3.1.1 `.data` segment (initialised data)

- `szClassName`: window class name (`"FileExplorerApp"`)
- `szAppName`: application title (`"Recursive File Explorer"`)
- `szEditClass`, `szButtonClass`, `szStaticClass`: Windows control class names
- `szOKButton`: button label (`"Browse"`)
- `szDefaultInput`: default path pre-filled in the TextBox (`"C:\Users\user"`)
- `searchPattern`: wildcard `\*.*` for file search

3.1.2 `.data?` segment (uninitialised data)

- `hInstance`, `hWnd`, `hEdit`, `hButton`, `hResult`: handles for the window and its controls
- `path`: user-entered path (260 bytes)
- `searchPath`: path + `\*.*` (520 bytes)
- `findData`: `WIN32_FIND_DATAA` structure (320 bytes)
- `stackPaths`: path stack ($60 \times 260 = 15\,600$ bytes)
- `stackDepths`: depth stack ($60 \times 4 = 240$ bytes)
- `resultBuffer`: 32 000-byte buffer that accumulates all results
- `resultPtr`: current write position within `resultBuffer`

3.2 General operation

Unlike the CLI version, which reads and writes directly to the console, the GUI version waits for the user to type a path in the dedicated text box and click **Browse**. The recursive search then works identically to the CLI, except for output: all results are accumulated in `resultBuffer` and sent to the results area in a single `SetWindowTextA` call.

3.3 Initialisation

On startup, `GetModuleHandleA` retrieves the current instance handle. The window class is registered via `RegisterClassExA` with the `WNDCLASSEX` structure filled in manually (style `CS_HREDRAW|CS_VREDRAW`, arrow cursor, `COLOR_BTNFACE` background). The main window (900 × 600 pixels) is created with `CreateWindowExA` and made visible with `ShowWindow`.

3.4 Control creation – `CreateControls`

When the `WM_CREATE` message is received, `WndProc` calls `CreateControls`, which creates three controls:

- **Path input TextBox (ID 1000)**: text entry for the directory path; the default path is injected immediately via `SetWindowTextA`.
- **"Browse" button (ID 1001)**: triggers the recursive scan.
- **Results area (ID 1002)**: multiline read-only TextBox with vertical and horizontal scroll bars.

All controls are created with `WS_CHILD | WS_VISIBLE` to be parented to the main window and immediately visible.

3.5 Triggering the scan – `ScanAndDisplay`

On a button click, `WndProc` receives `WM_COMMAND`. When the control ID is `1001`, `ScanAndDisplay` is called. This function:

1. Reads the TextBox content with `GetWindowTextA` (max 260 characters).
2. Resets the control variables (`resultPtr`, `fileCounter`, `currentDepth`) and both stacks.
3. Writes the root path header via `AddRootHeader`.
4. Pushes the initial path at depth 0.
5. Runs the main loop, which is identical to the CLI version.

3.6 Displaying results – `AddFileNameToResult` / `AddRootHeader`

`AddFileNameToResult` writes into `resultBuffer` using `resultPtr` as its cursor. It first inserts as many TAB characters (ASCII 9) as the value of `currentDepth`, producing hierarchical indentation. The filename (`cFileName` at offset 44 in `WIN32_FIND_DATAA`) is then copied byte-by-byte, followed by a CR LF newline.

Once the recursion is complete, `SetWindowTextA` sends the entire `resultBuffer` to the results area in one call. This contrasts with the CLI version, which writes line-by-line via `WriteFile`.

4 CLI vs GUI Comparison

Aspect	CLI	GUI
Path input	`ReadFile` (stdin)	`GetWindowTextA` (TextBox)
Result display	`WriteFile` line-by-line	`SetWindowTextA` single call
Depth tracking	None (no indentation)	Yes (`stackDepths` stack)
Result buffer	None (direct console output)	`resultBuffer` 32 000 bytes
Linker subsystem	`CONSOLE`	`WINDOWS`

5 Challenges encountered

1. `.obj` **file corruption**: in early iterations, accessing `dwFileAttributes` via a compile-time direct address caused the generated object file to disappear. The fix was to use the symbolic address `dword ptr findData`.
2. **Infinite loop on `.` and `..`**: without explicit filtering of these two entries, the stack would overflow immediately. The double-character test resolved the issue.
3. **Backslash handling**: when building the search path, a check was required to determine whether the user-supplied path already ended with \ before appending one, to avoid malformed patterns (`C:\\*.*`).
4. **Result buffer limit (GUI)**: `resultBuffer` is sized at 32 000 bytes. Guard comparisons (`cmp resultPtr, 31800`) prevent writes beyond the buffer boundary.

6 Conclusion

This project provided hands-on experience with x86 32-bit Windows assembly programming by interacting directly with the Win32 API. Both versions (CLI and GUI) demonstrate how the same recursive traversal algorithm can be adapted to two different interface paradigms while sharing the same iterative stack structure. The GUI version additionally introduces depth tracking for hierarchical visual indentation of the results.